

Vanita Vishram Women's University, Surat, Gujarat

Department of Information Technology

Academic Year: 2026–2027

Class: T.Y. B.Sc. IT

Div:-A

COMPUTER GRAPHICS PROJECT REPORT

ON

ANIMATED NEWTON'S CRADLE

COMPUTER GRAPHICS

Submitted By

Group Members

1. Name: Khunt Krupa Jagdishbhai

Roll No.:2426070

2. Name:Chavda Gunjan

Roll No.: 2426024

3. Name: Gohil Hiral

Roll No.: 2426036

4. Name: Gohil Kinjal

Roll No.: 2426037

5. Name: Kathiriya Liza

Roll No.: 2426062

INDEX

Sr. No.	Chapter / Topic	Page No.
1	Introduction	
2	Project Overview	
3	Problem Statement	
4	Objectives	
5	Scope of the Project	
6	Hardware Requirements	
7	Software Requirements	
8	Technologies Used	
9	Computer Graphics Concepts Used	
10	System Design	
11	Working of the Project	
12	Project Features	
13	Implementation	
14	Source Code Explanation	
15	Output Screenshots	
16	Advantages	
17	Limitations	
18	Future Scope	
19	Conclusion	
20	References	

1. INTRODUCTION

Computer Graphics is a field of computer science that deals with the creation, manipulation, and representation of visual images using computers. It is widely used in areas such as education, gaming, animation, engineering, simulation, entertainment, and scientific visualization.

The project "Animated Newton's Cradle" is a computer graphics-based animation project designed to demonstrate the motion and collision of spherical objects. Newton's Cradle is a mechanical device consisting of a series of identical metal balls suspended by strings from a frame.

When one ball is pulled to one side and released, it collides with the stationary balls. The force and momentum are transferred through the middle balls, causing the ball at the opposite end to move outward. This process continues repeatedly.

In this project, the physical behavior of Newton's Cradle is simulated using computer graphics programming. The project demonstrates concepts such as 2D graphics, object drawing, geometric transformations, motion, animation, collision simulation, and user interaction.

The main objective of this project is to provide an interactive visual representation of Newton's Cradle using computer graphics techniques.

2. PROJECT OVERVIEW

The Animated Newton's Cradle project is an interactive graphical application that simulates the movement of five metallic balls suspended from a wooden frame.

Main components include a graphical welcome screen, Newton's Cradle wooden stand, five metallic balls, suspended strings, ball movement animation, collision simulation, continuous animation, speed control, color customization, physics information panel, FPS display, user interaction, and exit/menu options.

The project visually demonstrates the transfer of motion and momentum between the balls and is designed to make the concept easier to understand and more engaging for students.

3. PROBLEM STATEMENT

Understanding physical concepts such as collision, momentum, and motion through theoretical explanations can sometimes be difficult for students.

Traditional diagrams are static and cannot effectively demonstrate continuous motion or collision between objects.

Therefore, the problem is to develop an interactive computer graphics application that visually simulates the working of Newton's Cradle and demonstrates the concepts of motion and collision through animation.

4. OBJECTIVES

1. To develop an interactive Computer Graphics application.

2. To create an animated simulation of Newton's Cradle.
3. To demonstrate the concept of motion and collision.
4. To understand 2D graphics programming.
5. To implement object movement using animation techniques.
6. To demonstrate geometric transformations.
7. To create realistic-looking metallic balls and a wooden frame.
8. To provide user controls for animation speed.
9. To provide an interactive graphical interface.
10. To understand practical applications of Computer Graphics.
11. To demonstrate the transfer of momentum visually.
12. To make the learning process more interactive and interesting.

5. SCOPE OF THE PROJECT

The scope of the Animated Newton's Cradle project includes the development of a graphical simulation that can be used for educational and demonstration purposes.

It can be used in Computer Graphics education, physics demonstrations, animation learning, programming laboratories, classroom presentations, and interactive learning applications.

The project can be extended in the future by adding advanced physics calculations, sound effects, 3D graphics, realistic lighting, and user-controlled simulation parameters.

6. HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or higher
- RAM: Minimum 4 GB
- Storage: Minimum 500 MB free space
- Display: Standard monitor
- Keyboard: Standard keyboard
- Mouse: Standard mouse

7. SOFTWARE REQUIREMENTS

- Operating System: Windows
- Programming Language: C/C++
- Graphics Library: graphics.h

- **Compiler:** Turbo C++ / Compatible C++ Compiler
- **IDE:** Turbo C++ or compatible development environment
- **Graphics Driver:** BGI Graphics Driver

8. TECHNOLOGIES USED

C/C++ Programming: C/C++ is used to implement the logic and functionality of the project. It provides programming constructs required to control the movement and animation of the objects.

graphics.h: The graphics.h library is used to create graphical objects and display them on the screen. It provides functions for drawing lines, circles, rectangles, polygons, setting colors, filling shapes, displaying text, and creating graphical animations.

Computer Animation: Animation is used to create the illusion of movement by continuously updating the positions of graphical objects.

9. COMPUTER GRAPHICS CONCEPTS USED

2D Graphics: The project uses two-dimensional coordinates to draw objects on the screen. The X-axis represents horizontal position and the Y-axis represents vertical position.

Geometric Primitives: Lines, circles, rectangles, and polygons are used. The balls are represented using circles, while the stand is created using lines and rectangles.

Translation: Translation is used to change the position of the balls. When a ball moves from one position to another, its X and Y coordinates are updated.

Animation: Animation is achieved by repeatedly drawing the object, updating its position, clearing the previous frame, and drawing the object at the new position.

Collision Simulation: The project simulates collision between Newton's Cradle balls. When the moving ball reaches the stationary balls, movement is transferred to the opposite side.

Double Buffering: Double buffering can be used to reduce flickering during animation by preparing the next frame before displaying it.

Color and Filling: Different colors and filling techniques are used to create metallic balls, wooden stand, background, buttons, and information panels.

Text Rendering: Text is displayed for project title, menu options, physics information, speed information, FPS information, and instructions.

10. SYSTEM DESIGN

1. **Welcome Screen:** Displays the project title and introduction.
2. **Main Menu:** Provides options to start animation, change settings, and exit.
3. **Newton's Cradle Display:** Displays the main animated cradle.

- 4. Animation Module: Controls movement of balls.**
- 5. Collision Module: Simulates transfer of motion.**
- 6. Speed Control Module: Allows animation speed control.**
- 7. Color Control Module: Allows customization of graphical objects.**
- 8. Physics Information Module: Displays basic information about momentum and collision.**
- 9. FPS Counter: Displays approximate frame rate.**
- 10. Exit Module: Closes the application safely.**

11. WORKING OF THE PROJECT

- Step 1: The program starts by initializing the graphics environment.**
- Step 2: A welcome screen is displayed with the project title.**
- Step 3: The wooden stand, strings, and five balls are created.**
- Step 4: The user selects an option from the main menu.**
- Step 5: When animation starts, one ball moves from one side.**
- Step 6: The moving ball collides with the stationary balls.**
- Step 7: Movement is transferred through the middle balls to the opposite end.**
- Step 8: The process continues repeatedly.**
- Step 9: The user can modify animation speed or settings.**
- Step 10: The user can exit the application.**

12. PROJECT FEATURES

- Animated Newton's Cradle**
- 3D-looking wooden stand**
- Metallic ball appearance**
- Physics information panel**
- Speed control**
- Color customization**
- FPS counter**
- Interactive menu**
- Optional collision sound**
- Welcome and Thank You screens**

13. IMPLEMENTATION

The project is implemented using C/C++ programming and the graphics.h library.

The implementation consists of graphics initialization, stand drawing, ball drawing, string drawing, animation logic, collision logic, menu system, and speed control.

The stand is created using lines, rectangles, and polygons. The five balls are created using circle functions. Lines represent suspension strings. Ball coordinates are continuously updated to create movement, and collision logic controls the transfer of motion.

14. SOURCE CODE

Header Files: Header files provide graphics and input/output functions. Example: #include <graphics.h>, #include <conio.h>, #include <stdio.h>, and #include <dos.h>.

Graphics Initialization: The graphics mode is initialized before displaying the graphical interface.

Drawing the Stand: Functions such as line(), rectangle(), bar(), setcolor(), setfillstyle(), and floodfill() can be used.

Drawing the Balls: The balls are drawn using the circle() function, with additional graphical effects for a metallic appearance.

Animation Logic: Ball coordinates are continuously updated using a loop, with the screen refreshed to create movement.

Collision Logic: The program checks ball positions and changes movement according to the animation logic.

Menu System: A graphical menu allows users to select options.

Speed Control: The delay between animation frames is modified to control speed.

```
/*=====
  ANIMATED NEWTON'S CRADLE WITH NEON THEME & LIVE TELEMETRY
  Department of Information Technology | VVWU
  =====*/
```

```
#include <graphics.h>
#include <conio.h>
#include <dos.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
```

```
#define PI 3.14159265
#define BALLS 5
#define SCREEN_W 640
#define SCREEN_H 480
```

```

/* Global Settings */
int ballRadius = 14;
int topY = 65;
int stringLength = 100;
int ballX[BALLS] = {240, 270, 300, 330, 360};

int animationDelay = 15;
int paused = 0;

/* Active Swing Mode: 1, 2, or 3 Balls */
int swingMode = 1;

/* Dynamic Theme Variables (0: Classic, 1: Cyberpunk/Neon) */
int currentTheme = 0;
int bgColor = BLACK;
int frameColor = YELLOW;
int ballColor = LIGHTBLUE;
int panelBg = BLACK;
int textColor = WHITE;
int accentColor = LIGHTCYAN;

/* Ball Color Customization List */
int ballColorsList[] = {LIGHTBLUE, LIGHTGREEN, LIGHTMAGENTA, LIGHTRED,
YELLOW, CYAN};
int colorIndex = 0;

/* Physics Values */
float gravity = 9.8;
float ballMass = 1.0;
float currentAngle = 0.0;
float currentVelocity = 0.0;
float currentMomentum = 0.0;
float currentEnergy = 0.0;
float currentPE = 0.0;
float currentKE = 0.0;

/* Oscilloscope Buffer Variables */
int scopeX = 40;
int lastScopeY = 365;

/* Apply Color Palette Based on Active Theme */
void applyThemePalette()
{
    if (currentTheme == 0)
    {
        /* CLASSIC LAB MODE */
        bgColor = BLACK;
    }
}

```

```

        frameColor = YELLOW;
        ballColor = LIGHTBLUE;
        panelBg = BLACK;
        textColor = WHITE;
        accentColor = LIGHTCYAN;
    }
    else
    {
        /* CYBERPUNK / NEON NIGHT MODE */
        bgColor = DARKGRAY;      /* Midnight / Dark purple backdrop */
        frameColor = CYAN;       /* Neon Cyan Frame */
        ballColor = LIGHTGREEN;  /* Neon Green Balls */
        panelBg = BLUE;          /* Deep Blue Dashboard */
        textColor = YELLOW;      /* Neon Yellow Text */
        accentColor = LIGHTMAGENTA; /* Neon Pink Accents */
    }
    colorIndex = 0; // Reset color index on theme change
}

void toggleTheme()
{
    currentTheme = !currentTheme;
    applyThemePalette();
}

/* Function to Cycle Through Ball Colors */
void cycleBallColor()
{
    colorIndex = (colorIndex + 1) % 6;
    ballColor = ballColorsList[colorIndex];
}

/* Sound Effect Function for Metal Collision */
void playCollisionSound()
{
    int freq = 700 + (swingMode * 150);
    sound(freq);
    delay(15);
    nosound();
}

/* Dynamic Impact Spark Effect Drawing Function */
void drawImpactSpark(int impactX, int impactY)
{
    setcolor(YELLOW);
    circle(impactX, impactY, 3);
    setfillstyle(SOLID_FILL, WHITE);
    fillellipse(impactX, impactY, 2, 2);
}

```

```

/* Energy Wave Sparks radiating outward */
setcolor(LIGHTRED);
line(impactX - 10, impactY - 10, impactX - 3, impactY - 3);
line(impactX + 10, impactY - 10, impactX + 3, impactY - 3);
line(impactX - 10, impactY + 10, impactX - 3, impactY + 3);
line(impactX + 10, impactY + 10, impactX + 3, impactY + 3);

setcolor(YELLOW);
line(impactX, impactY - 12, impactX, impactY - 4);
line(impactX, impactY + 12, impactX, impactY + 4);
line(impactX - 12, impactY, impactX - 4, impactY);
line(impactX + 12, impactY, impactX + 4, impactY);
}

/* Utility Functions */
void clearScreen()
{
    cleardevice();
    setbkcolor(bgColor);
}

void centerText(char *text, int y, int size, int color)
{
    int x;
    setcolor(color);
    settextstyle(DEFAULT_FONT, HORIZ_DIR, size);
    x = (SCREEN_W - textwidth(text)) / 2;
    outtextxy(x, y, text);
}

void drawVWULogo()
{
    setcolor(frameColor);
    circle(320, 55, 32);
    circle(320, 55, 25);

    /* Graduation cap */
    line(303, 50, 320, 40);
    line(320, 40, 337, 50);
    line(303, 50, 320, 60);
    line(320, 60, 337, 50);

    /* Book */
    line(310, 68, 320, 63);
    line(320, 63, 330, 68);

    settextstyle(DEFAULT_FONT, HORIZ_DIR, 1);

```

```

    outtextxy(307, 82, "VWU");
}

void welcomeScreen()
{
    applyThemePalette();
    clearScreen();

    setcolor(frameColor);
    rectangle(10, 10, 630, 470);
    rectangle(15, 15, 625, 465);

    drawVWULogo();

    centerText("VANITA VISHRAM WOMEN UNIVERSITY", 110, 2, frameColor);
    centerText("DEPARTMENT OF INFORMATION TECHNOLOGY", 145, 1, accentColor);
    centerText("COMPUTER GRAPHICS PROJECT", 175, 1, textColor);

    centerText("ANIMATED NEWTON'S CRADLE", 220, 3, LIGHTGREEN);
    centerText("Theme Toggle, Sparks, Multi-Ball & Telemetry", 265, 1,
LIGHTMAGENTA);

    setcolor(textColor);
    settextstyle(DEFAULT_FONT, HORIZ_DIR, 1);
    outtextxy(245, 310, "Developed By");

    outtextxy(245, 335, "Krupa");
    outtextxy(245, 350, "Gunjan");
    outtextxy(245, 365, "Hiral");
    outtextxy(245, 380, "Kinjal");
    outtextxy(245, 395, "Liza");

    setcolor(frameColor);
    outtextxy(185, 440, "Press any key to start the simulation");

    getch();
}

void calculatePhysics(int angle)
{
    float radians;
    float height;

    currentAngle = (float)abs(angle);
    radians = currentAngle * PI / 180.0;

    height = (stringLength / 100.0) * (1.0 - cos(radians));
}

```

```

currentVelocity = sqrt(2.0 * gravity * height);
currentMomentum = (ballMass * swingMode) * currentVelocity;

currentPE = (ballMass * swingMode) * gravity * height;
currentEnergy = (ballMass * swingMode) * gravity * ((stringLength /
100.0) * (1.0 - cos(40.0 * PI / 180.0)));

if (currentEnergy < currentPE)
    currentEnergy = currentPE + 0.1;

currentKE = currentEnergy - currentPE;
}

/* ----- DYNAMIC SCIENTIST DRAWING ----- */

void drawScientist(int x, int y, int handX, int handY, int isRightSide, int
isHolding)
{
    setcolor(textColor);

    /* Head & Glasses */
    circle(x, y, 10);
    circle(x - 3, y - 2, 1);
    circle(x + 3, y - 2, 1);
    arc(x, y + 2, 200, 340, 4);

    rectangle(x - 6, y - 5, x - 1, y);
    rectangle(x + 1, y - 5, x + 6, y);
    line(x - 1, y - 3, x + 1, y - 3);

    /* Lab Coat Body */
    rectangle(x - 8, y + 10, x + 8, y + 40);

    /* Legs */
    line(x - 4, y + 40, x - 6, y + 55);
    line(x + 4, y + 40, x + 6, y + 55);

    if (!isRightSide)
    {
        /* LEFT SCIENTIST */
        line(x - 8, y + 18, x - 16, y + 28);

        if (isHolding)
        {
            setcolor(frameColor);
            line(x + 8, y + 18, handX, handY);
            circle(handX, handY, 2);
        }
    }
}

```

```

        else
        {
            setcolor(textColor);
            line(x + 8, y + 18, x + 18, y + 28);
        }
    }
    else
    {
        /* RIGHT SCIENTIST */
        line(x + 8, y + 18, x + 16, y + 28);

        if (isHolding)
        {
            setcolor(frameColor);
            line(x - 8, y + 18, handX, handY);
            circle(handX, handY, 2);
        }
        else
        {
            setcolor(textColor);
            line(x + 8, y + 18, x - 18, y + 28);
        }
    }
}

/* ----- REAL-TIME LIVE MONITOR PANEL ----- */

void drawLiveMonitorPanel(int rawAngle)
{
    char str[50];
    int keWidth, peWidth;
    int currentScopeY;

    /* Monitor Outer Frame */
    setcolor(frameColor);
    rectangle(10, 275, 630, 465);
    setfillstyle(SOLID_FILL, frameColor);
    bar(10, 275, 630, 295);

    setcolor(bgColor == BLACK ? BLACK : WHITE);
    settextstyle(DEFAULT_FONT, HORIZ_DIR, 1);
    outtextxy(20, 282, "[ LIVE TELEMETRY DASHBOARD ]");

    sprintf(str, "MODE: %d BALL(S) | THEME: %s", swingMode, currentTheme ==
0 ? "CLASSIC" : "NEON");
    outtextxy(350, 282, str);

    /* Panel Background */

```

```

setfillstyle(SOLID_FILL, panelBg);
bar(11, 296, 629, 464);

/* 1. Oscilloscope */
setcolor(frameColor);
rectangle(20, 305, 250, 425);
line(20, 365, 250, 365);

setcolor(accentColor);
outtextxy(25, 310, "OSCILLOSCOPE WAVE");

currentScopeY = 365 - (int)(rawAngle * 1.1);

setcolor(LIGHTGREEN);
circle(scopeX, currentScopeY, 1);
line(scopeX - 1, lastScopeY, scopeX, currentScopeY);

lastScopeY = currentScopeY;
scopeX += 2;
if (scopeX > 248) {
    scopeX = 22;
    setfillstyle(SOLID_FILL, panelBg);
    bar(21, 320, 249, 424);
    setcolor(frameColor);
    line(20, 365, 250, 365);
    lastScopeY = currentScopeY;
}

/* 2. Digital Readouts */
setcolor(frameColor);
rectangle(260, 305, 440, 425);
outtextxy(268, 310, "[ DATA READINGS ]");

setcolor(textColor);
sprintf(str, "Angle : %d deg", rawAngle);
outtextxy(268, 330, str);

sprintf(str, "Vel   : %.2f m/s", currentVelocity);
outtextxy(268, 350, str);

sprintf(str, "Mom.   : %.2f kgM/s", currentMomentum);
outtextxy(268, 370, str);

sprintf(str, "Energy: %.2f J", currentEnergy);
outtextxy(268, 390, str);

/* 3. Gauges */
setcolor(accentColor);

```

```

rectangle(450, 305, 620, 425);
outtextxy(458, 310, "[ ENERGY GAUGES ]");

keWidth = (int)((currentKE / (currentEnergy > 0 ? currentEnergy : 1.0))
* 130);
if(keWidth < 0) keWidth = 0;
if(keWidth > 130) keWidth = 130;

setcolor(textColor);
outtextxy(458, 335, "KE:");
setfillstyle(SOLID_FILL, LIGHTRED);
bar(485, 335, 485 + keWidth, 347);
rectangle(485, 335, 610, 347);

peWidth = 130 - keWidth;
if(peWidth < 0) peWidth = 0;
if(peWidth > 130) peWidth = 130;

outtextxy(458, 365, "PE:");
setfillstyle(SOLID_FILL, LIGHTBLUE);
bar(485, 365, 485 + peWidth, 377);
rectangle(485, 365, 610, 377);

setcolor(LIGHTGREEN);
outtextxy(458, 400, "VERIFIED: E=KE+PE");

/* Controls Guide */
setcolor(textColor);
outtextxy(15, 445, "[1/2/3] Balls | [T] Theme | [C] Color | [P] Pause |
[R] Reset");
}

/* ----- ANIMATION RENDERING ----- */

void drawStand()
{
    setfillstyle(SOLID_FILL, BROWN);
    bar(150, 225, 450, 240);
    setcolor(frameColor);
    rectangle(150, 225, 450, 240);

    setfillstyle(SOLID_FILL, BROWN);
    bar(170, 65, 180, 225);
    bar(420, 65, 430, 225);
    bar(170, 58, 430, 68);

    setfillstyle(SOLID_FILL, LIGHTGRAY);
    fillellipse(175, 63, 2, 2);

```

```

    fillellipse(425, 63, 2, 2);
}

void drawBall(int x, int angle, int *ballPX, int *ballPY)
{
    int px, py;

    px = x + (int)(stringLength * sin(angle * PI / 180.0));
    py = topY + (int)(stringLength * cos(angle * PI / 180.0));

    if (ballPX) *ballPX = px;
    if (ballPY) *ballPY = py;

    calculatePhysics(angle);

    setcolor(WHITE);
    line(x, topY, px, py);

    setfillstyle(SOLID_FILL, DARKGRAY);
    fillellipse(px + 2, 233, ballRadius - 4, 3);

    setfillstyle(SOLID_FILL, ballColor);
    fillellipse(px, py, ballRadius, ballRadius);

    setcolor(WHITE);
    circle(px, py, ballRadius);
    setfillstyle(SOLID_FILL, WHITE);
    fillellipse(px - 4, py - 4, 2, 2);
}

void drawHeader()
{
    setcolor(frameColor);
    settextstyle(DEFAULT_FONT, HORIZ_DIR, 2);
    outtextxy(140, 12, "NEWTON'S CRADLE SIMULATOR");

    setcolor(accentColor);
    settextstyle(DEFAULT_FONT, HORIZ_DIR, 1);
    outtextxy(150, 35, "Multi-Ball Dynamics & Telemetry Dashboard");
}

void drawLeftSwing(int angle, int isLeftScientistHolding, int isSparkActive)
{
    int i, bX, bY;
    clearScreen();

    drawHeader();
    drawStand();

```

```

/* Render Swinging Balls on Left */
for (i = 0; i < swingMode; i++)
{
    if (i == 0)
        drawBall(ballX[i], angle, &bX, &bY);
    else
        drawBall(ballX[i], angle, NULL, NULL);
}

/* Render Stationary Balls */
for (i = swingMode; i < BALLS; i++)
{
    drawBall(ballX[i], 0, NULL, NULL);
}

/* Trigger Impact Spark Effect if active */
if (isSparkActive)
{
    drawImpactSpark(ballX[swingMode] - ballRadius, topY + stringLength);
}

/* Left Scientist interacts with the leftmost moving ball */
drawScientist(110, 140, bX - 5, bY, 0, isLeftScientistHolding);

/* Right Scientist Idle */
drawScientist(490, 140, 472, 168, 1, 0);

drawLiveMonitorPanel(angle);
}

void drawRightSwing(int angle, int isRightScientistReceiving, int
isSparkActive)
{
    int i, bX, bY;
    clearScreen();

    drawHeader();
    drawStand();

    /* Left Scientist Idle */
    drawScientist(110, 140, 128, 168, 0, 0);

    /* Render Stationary Balls */
    for (i = 0; i < BALLS - swingMode; i++)
    {
        drawBall(ballX[i], 0, NULL, NULL);
    }
}

```

```

/* Render Swinging Balls on Right */
for (i = BALLS - swingMode; i < BALLS; i++)
{
    if (i == BALLS - 1)
        drawBall(ballX[i], angle, &bX, &bY);
    else
        drawBall(ballX[i], angle, NULL, NULL);
}

/* Trigger Impact Spark Effect if active */
if (isSparkActive)
{
    drawImpactSpark(ballX[BALLS - swingMode - 1] + ballRadius, topY +
stringLength);
}

/* Right Scientist interacts with the rightmost moving ball */
drawScientist(490, 140, bX + 5, bY, 1, isRightScientistReceiving);

drawLiveMonitorPanel(angle);
}

void resetAnimation()
{
    currentAngle = 0;
    currentVelocity = 0;
    currentMomentum = 0;
    currentEnergy = 0;
    currentKE = 0;
    currentPE = 0;
    paused = 0;
    scopeX = 22;
}

int checkAnimationKeys()
{
    char ch;

    if (kbhit())
    {
        ch = getch();

        if (ch == 27)
            return 0;

        if (ch == '1') { swingMode = 1; resetAnimation(); }
        if (ch == '2') { swingMode = 2; resetAnimation(); }
    }
}

```

```

    if (ch == '3') { swingMode = 3; resetAnimation(); }

    if (ch == 't' || ch == 'T')
        toggleTheme();

    /* Ball Color Change Shortcut */
    if (ch == 'c' || ch == 'C')
        cycleBallColor();

    if (ch == 'p' || ch == 'P')
        paused = !paused;

    if (ch == 'r' || ch == 'R')
        resetAnimation();

    if (ch == '+')
    {
        if (animationDelay > 3)
            animationDelay -= 2;
    }

    if (ch == '-')
        animationDelay += 2;
}

return 1;
}

/* ----- MAIN SIMULATION ENGINE ----- */

void runAnimation()
{
    int i;
    resetAnimation();

    /* INITIAL SETUP: LEFT SCIENTIST PULLS BALLS BACK (-40 deg) */
    for (i = 0; i >= -40; i--)
    {
        if (!checkAnimationKeys()) return;
        drawLeftSwing(i, 1, 0);
        delay(25);
    }

    delay(400);

    /* INFINITE CRADLE CYCLE WITH THEME TOGGLE & SPARKS */
    while (1)
    {

```

```

/* 1. Left Balls swing down to center */
for (i = -40; i <= 0; i++)
{
    if (!checkAnimationKeys()) return;
    while (paused) {
        drawLeftSwing(i, 0, 0);
        if (!checkAnimationKeys()) return;
        delay(50);
    }

    /* Trigger Spark Effect at exact center collision frame */
    if (i == 0)
    {
        drawLeftSwing(i, 0, 1);
        playCollisionSound();
        delay(30);
    }
    else
    {
        drawLeftSwing(i, 0, 0);
    }
    delay(animationDelay);
}

/* 2. Right Balls swing outward */
for (i = 0; i <= 40; i++)
{
    if (!checkAnimationKeys()) return;
    while (paused) {
        drawRightSwing(i, (i > 25), 0);
        if (!checkAnimationKeys()) return;
        delay(50);
    }
    drawRightSwing(i, (i > 25), 0);
    delay(animationDelay);
}

/* 3. Right Balls return to center */
for (i = 40; i >= 0; i--)
{
    if (!checkAnimationKeys()) return;
    while (paused) {
        drawRightSwing(i, (i > 25), 0);
        if (!checkAnimationKeys()) return;
        delay(50);
    }

    /* Trigger Spark Effect at exact center collision frame */

```

```

        if (i == 0)
        {
            drawRightSwing(i, (i > 25), 1);
            playCollisionSound();
            delay(30);
        }
        else
        {
            drawRightSwing(i, (i > 25), 0);
        }
        delay(animationDelay);
    }

    /* 4. Left Balls swing outward */
    for (i = 0; i >= -40; i--)
    {
        if (!checkAnimationKeys()) return;
        while (paused) {
            drawLeftSwing(i, (i < -25), 0);
            if (!checkAnimationKeys()) return;
            delay(50);
        }
        drawLeftSwing(i, (i < -25), 0);
        delay(animationDelay);
    }
}

/* ----- MENUS & SCREENS ----- */

void educationalScreen()
{
    clearScreen();
    setcolor(frameColor);
    rectangle(10, 10, 630, 470);

    centerText("NEWTON'S CRADLE - EDUCATIONAL MODE", 35, 2, frameColor);

    setcolor(textColor);
    settextstyle(DEFAULT_FONT, HORIZ_DIR, 1);

    outtextxy(60, 100, "1. Demonstrates conservation of momentum and
energy.");
    outtextxy(60, 140, "2. Toggle Theme ('T') switches Classic & Cyberpunk
Neon UI.");
    outtextxy(60, 180, "3. Change Ball Color ('C') cycles through custom
neon colors.");
}

```

```

    outtextxy(60, 220, "4. Dynamic Impact Sparks highlight energy transfer
at collision.");
    outtextxy(60, 260, "5. Multi-Ball Modes (1, 2, 3) simulate variable mass
swing.");
    outtextxy(60, 300, "6. Real-time telemetry dashboard tracks Live Energy
distribution.");

    setcolor(frameColor);
    outtextxy(210, 430, "Press any key to continue...");
    getch();
}

void aboutProject()
{
    clearScreen();
    setcolor(frameColor);
    rectangle(10, 10, 630, 470);

    centerText("ABOUT THE PROJECT", 40, 3, LIGHTGREEN);

    setcolor(textColor);
    settextstyle(DEFAULT_FONT, HORIZ_DIR, 1);

    outtextxy(70, 110, "Project Name : Animated Newton's Cradle (Neon &
Telemetry)");
    outtextxy(70, 145, "Technology      : C / C++ Graphics & Sound (graphics.h
/ dos.h)");
    outtextxy(70, 180, "Domain          : Computer Graphics & Interactive
Simulation");
    outtextxy(70, 215, "Concept          : Momentum & Energy Conservation");
    outtextxy(70, 250, "University       : Vanita Vishram Women University");
    outtextxy(70, 285, "Department      : Information Technology");

    outtextxy(70, 350, "Team Members:");
    outtextxy(100, 380, "Krupa | Gunjan | Hiral | Kinjal | Liza");

    setcolor(frameColor);
    outtextxy(210, 430, "Press any key to continue...");
    getch();
}

int mainMenu()
{
    char choice;

    while (1)
    {
        clearScreen();

```

```

setcolor(frameColor);
rectangle(10, 10, 630, 470);
rectangle(15, 15, 625, 465);

centerText("ANIMATED NEWTON'S CRADLE", 40, 3, LIGHTGREEN);
centerText("MAIN MENU", 85, 2, accentColor);

setcolor(textColor);
settextstyle(DEFAULT_FONT, HORIZ_DIR, 1);

outtextxy(180, 120, "1. Start Interactive Simulation");
outtextxy(180, 150, "2. Educational Mode");
outtextxy(180, 180, "3. About Project");
outtextxy(180, 210, "4. Toggle Theme (Classic / Neon)");
outtextxy(180, 240, "5. Change Ball Color");
outtextxy(180, 270, "6. Increase Speed");
outtextxy(180, 300, "7. Decrease Speed");
outtextxy(180, 330, "8. Exit");

setcolor(frameColor);
outtextxy(170, 385, "Select an option (1-8):");

choice = getch();

if (choice == '1')
    return 1;
if (choice == '2')
    educationalScreen();
if (choice == '3')
    aboutProject();
if (choice == '4')
    toggleTheme();
if (choice == '5')
    cycleBallColor();
if (choice == '6')
{
    if (animationDelay > 3)
        animationDelay -= 2;
}
if (choice == '7')
    animationDelay += 2;
if (choice == '8' || choice == 27)
    return 0;
}
}

```

```

void thankYouScreen()

```

```

{
    clearScreen();

    setcolor(frameColor);
    rectangle(10, 10, 630, 470);
    rectangle(15, 15, 625, 465);

    centerText("THANK YOU", 55, 4, LIGHTGREEN);
    centerText("FOR WATCHING OUR PROJECT", 120, 2, textColor);

    centerText("ANIMATED NEWTON'S CRADLE", 175, 2, accentColor);

    setcolor(LIGHTMAGENTA);
    settextstyle(DEFAULT_FONT, HORIZ_DIR, 1);

    outtextxy(170, 240, "Key Features Demonstrated:");
    outtextxy(190, 275, "* Cyberpunk / Neon Interactive Theme Toggle");
    outtextxy(190, 300, "* Dynamic Ball Color Customization ('C' Key)");
    outtextxy(190, 325, "* Dynamic Collision Impact Spark Wave");
    outtextxy(190, 350, "* Multi-Ball Modes (1, 2, 3 Balls)");
    outtextxy(190, 375, "* Dual Scientists & Real-Time Telemetry");

    setcolor(frameColor);
    outtextxy(170, 425, "Vanita Vishram Women University");

    setcolor(textColor);
    outtextxy(220, 450, "Press any key to exit");

    getch();
}

void main()
{
    int gd = DETECT, gm;
    int menuResult;

    initgraph(&gd, &gm, "C:\\\\TURBOC3\\\\BGI");

    welcomeScreen();
    menuResult = mainMenu();

    if (menuResult == 1)
    {
        runAnimation();
    }

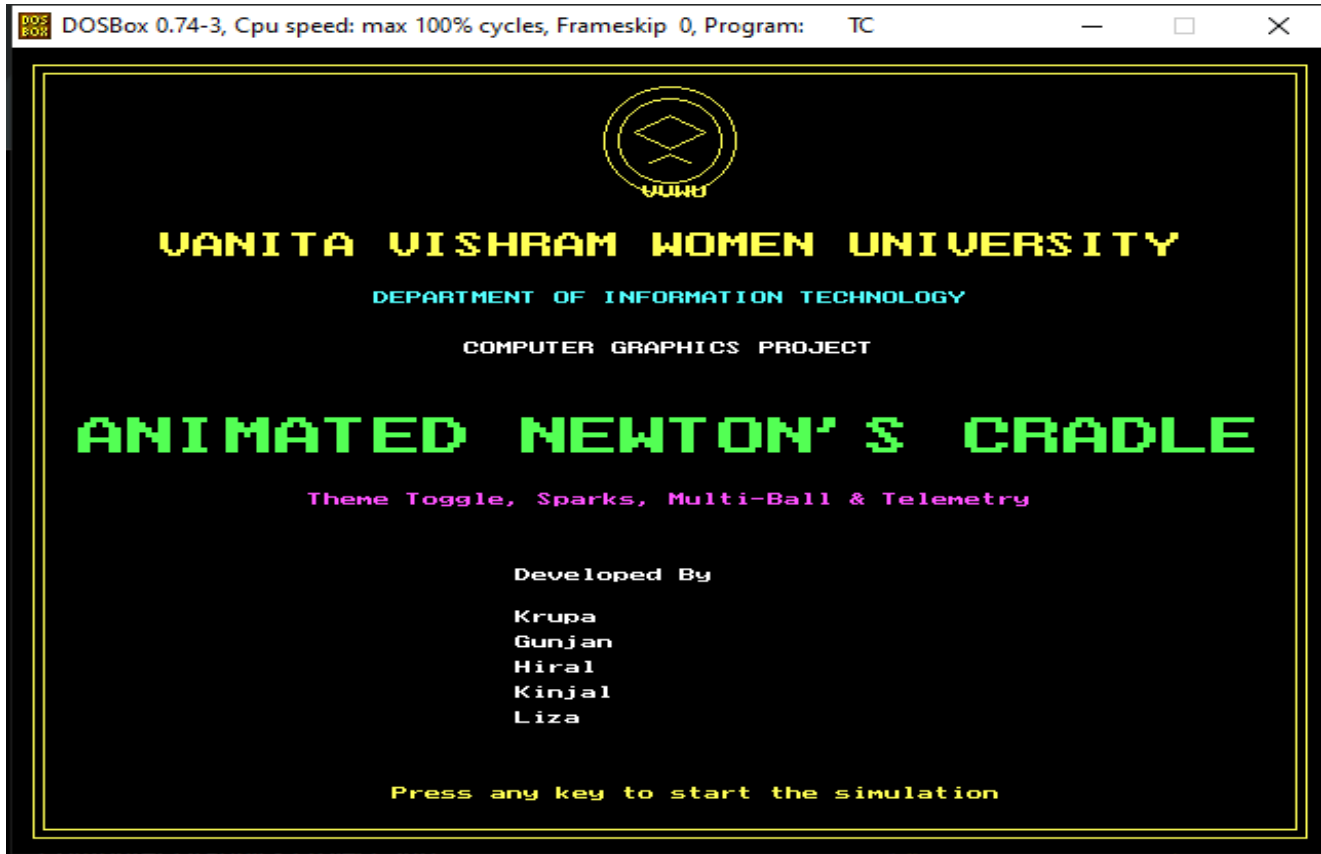
    thankYouScreen();
    closegraph();
}

```

}

15. OUTPUT SCREENSHOTS

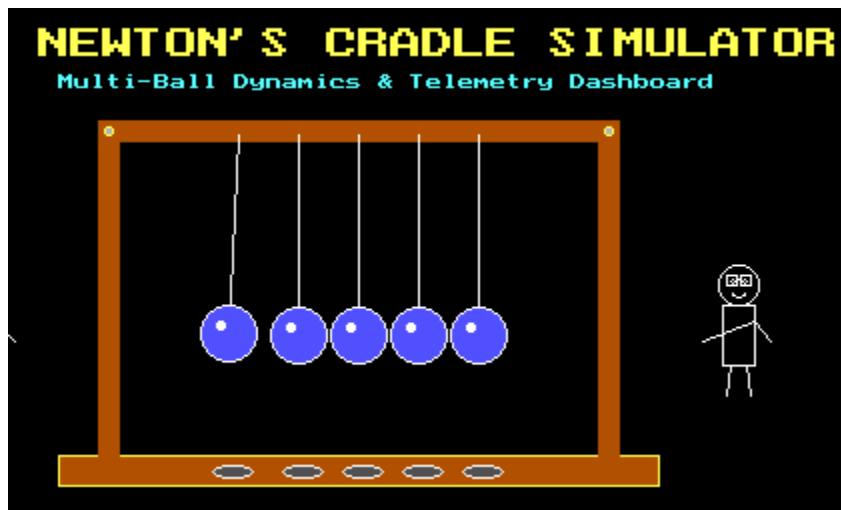
Screenshot 1: Welcome Screen



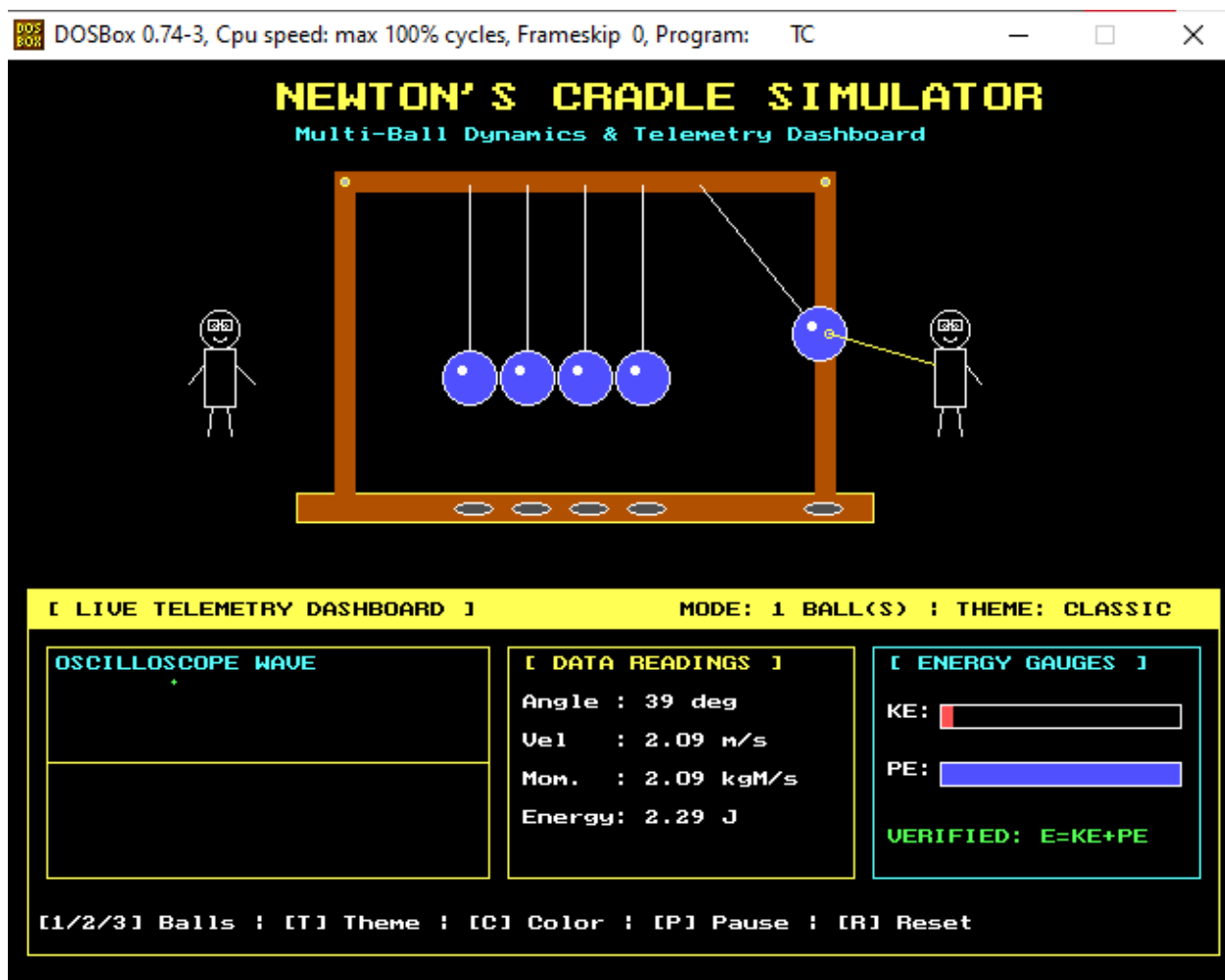
Screenshot 2: Main Menu



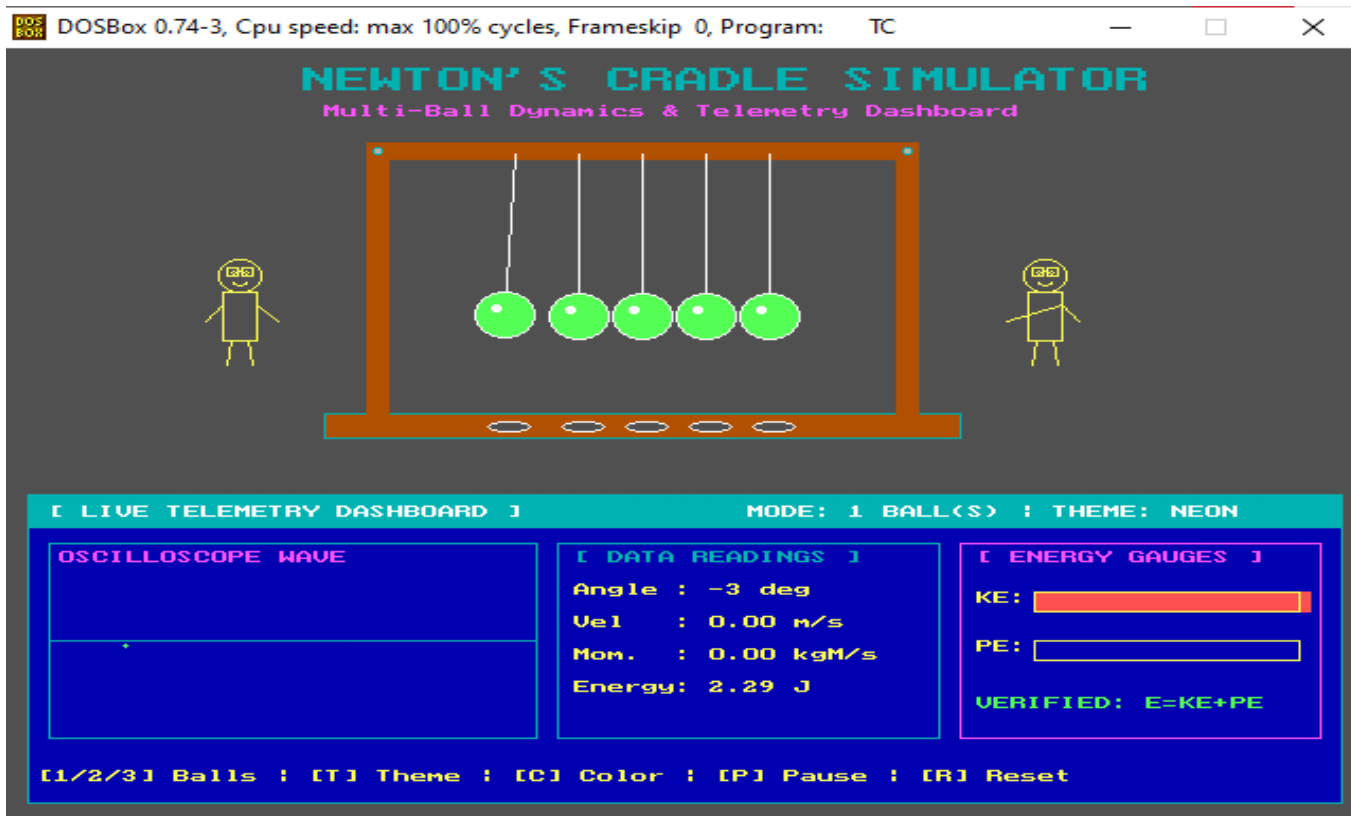
Screenshot 3: Newton's Cradle Stand



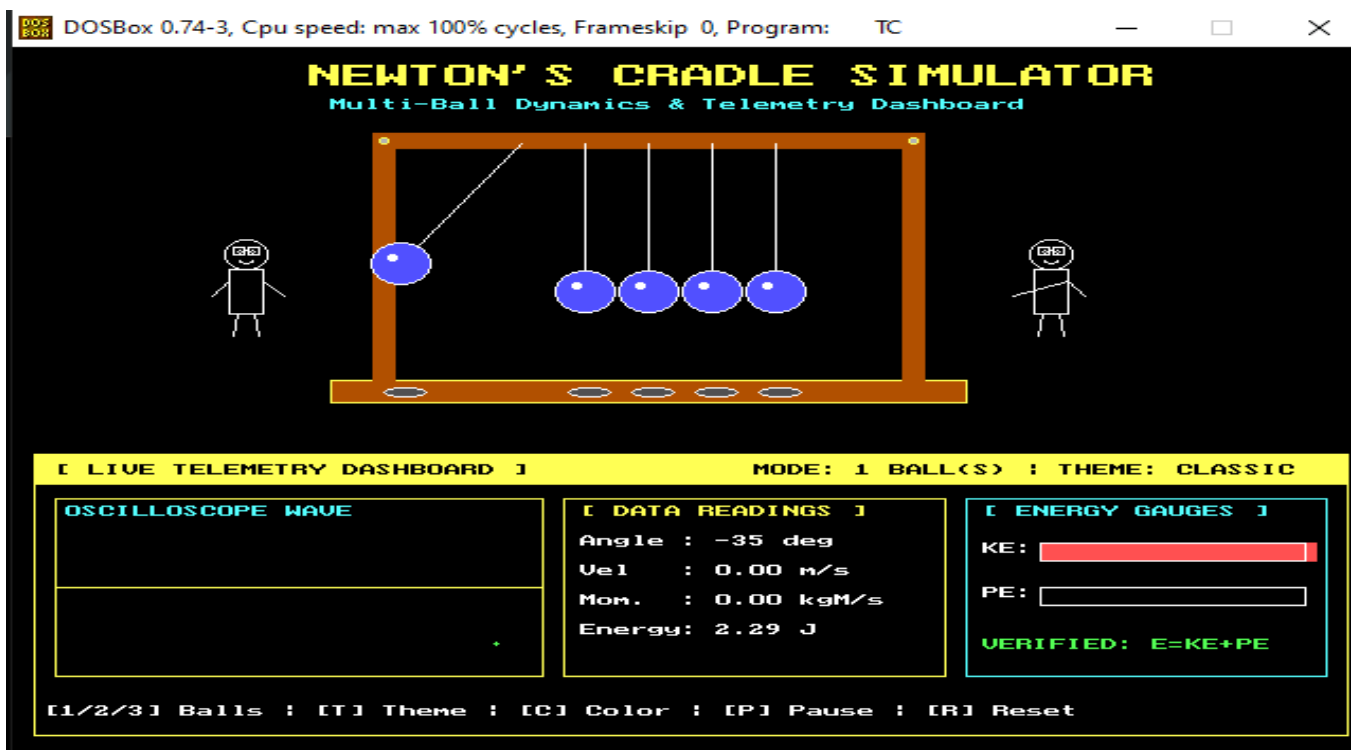
Screenshot 4: Animation – Left Ball in Motion



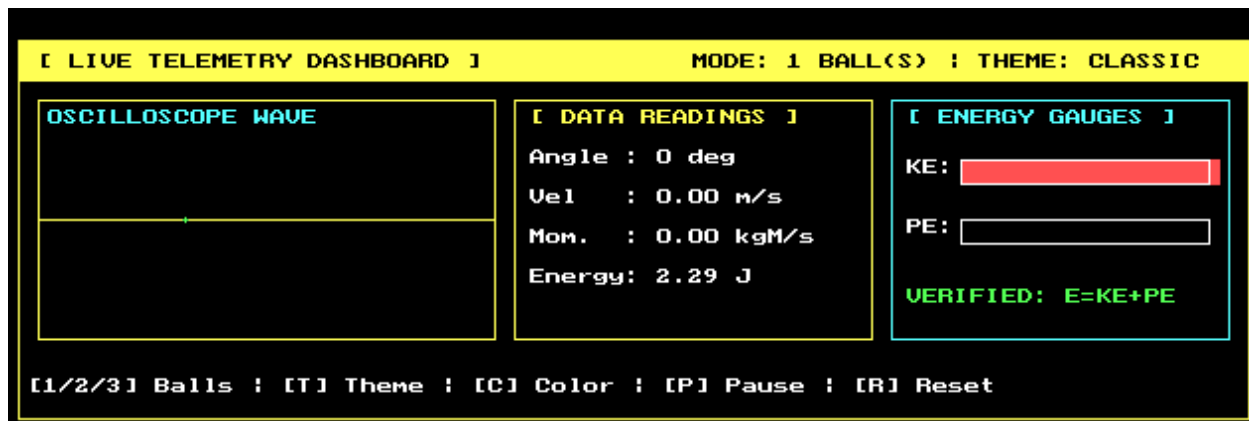
Screenshot 5: theme color change



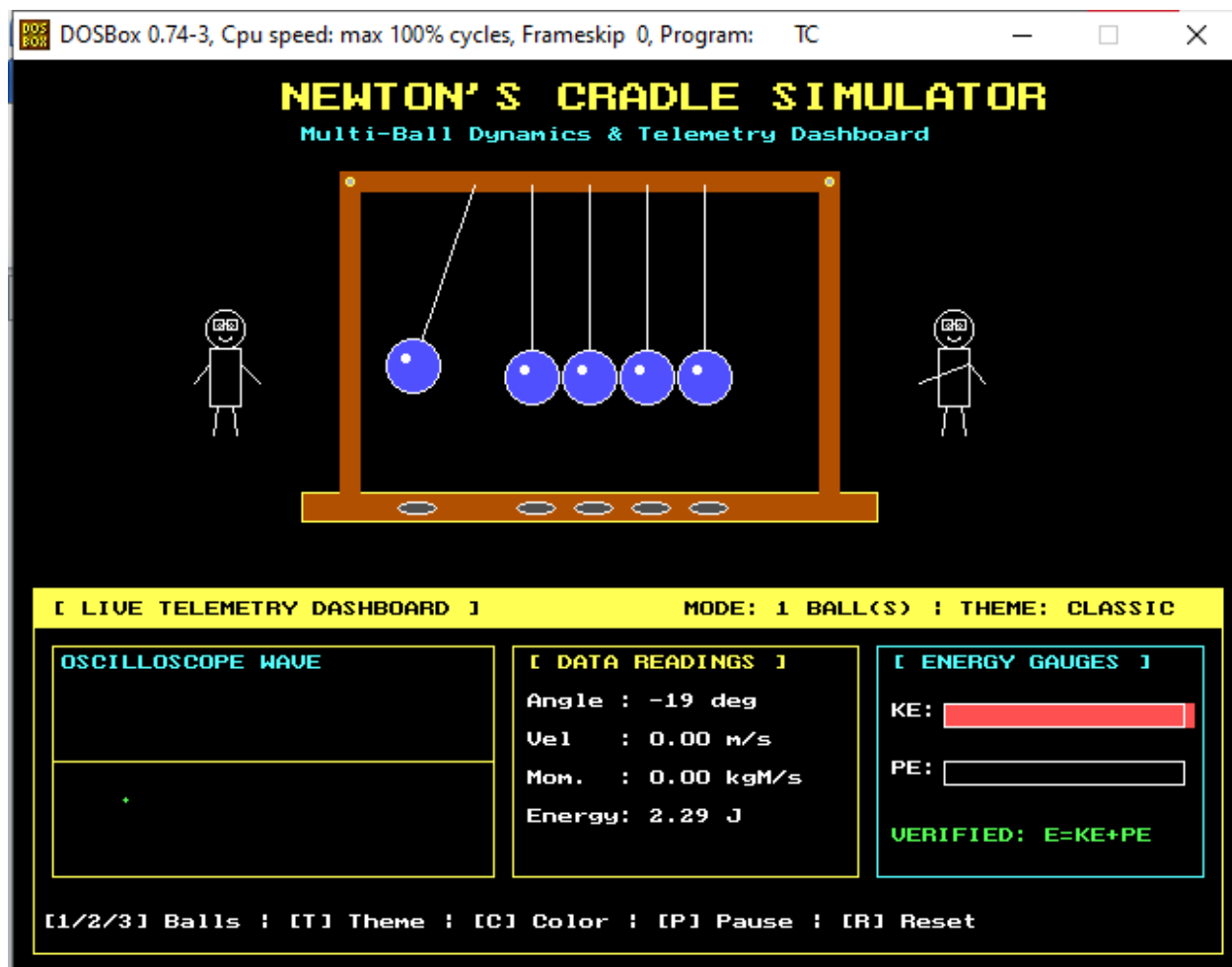
Screenshot 6: Opposite Ball Movement



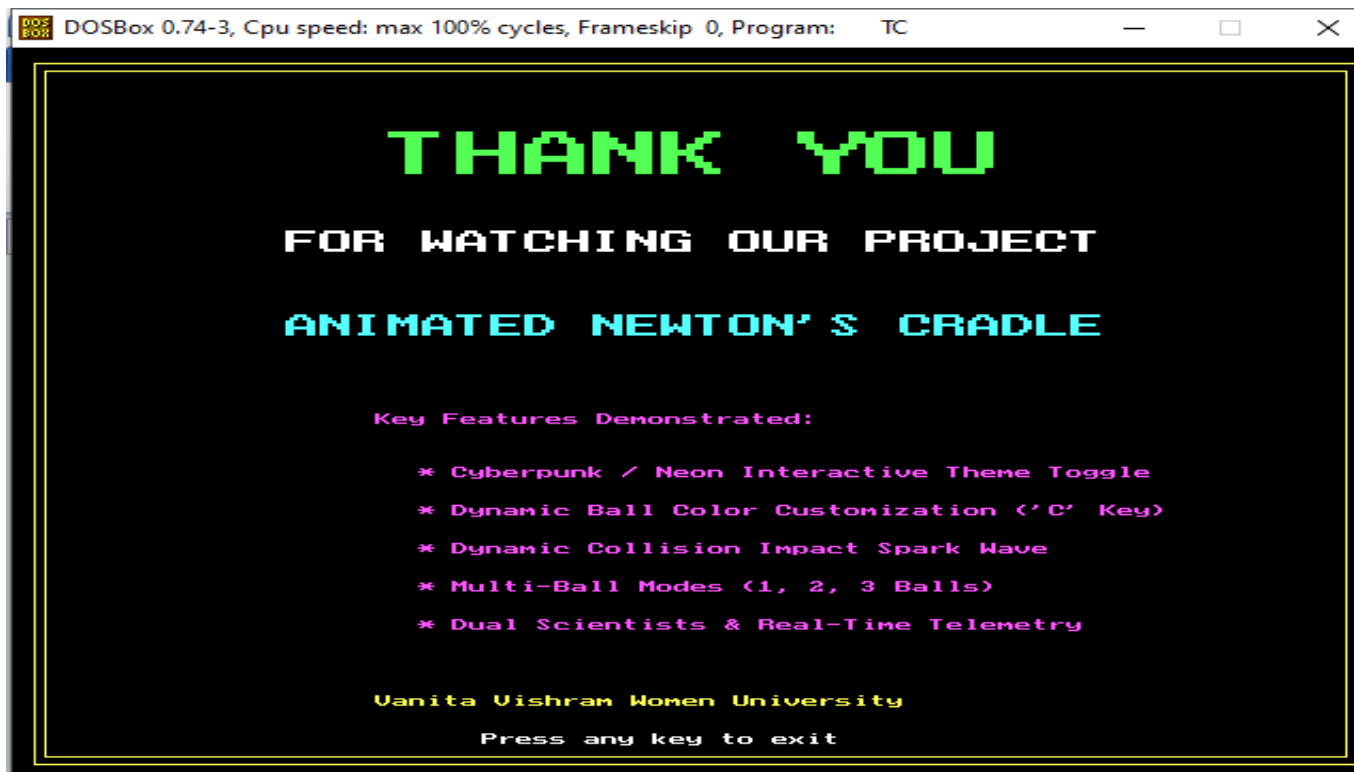
Screenshot 7: Physics Information Panel



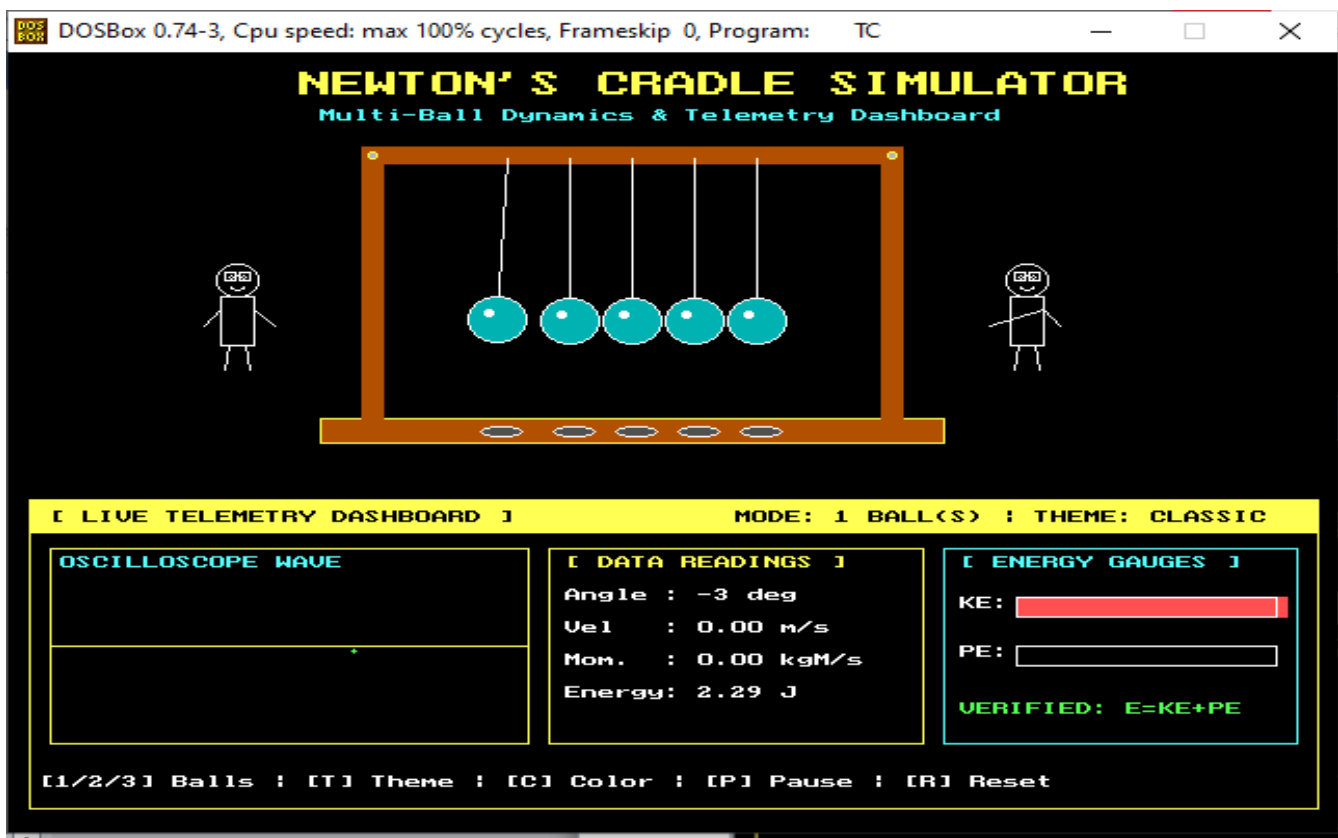
Screenshot 8: stop motion



Screenshot 9: Final / Thank You Screen



Screenshot 10: ball color changes



16. ADVANTAGES

- 1. Provides a visual understanding of motion and collision.**
- 2. Demonstrates Computer Graphics concepts practically.**
- 3. Makes learning more interactive.**
- 4. Provides real-time animation.**
- 5. Simple and easy to understand.**
- 6. Useful for educational demonstrations.**
- 7. Demonstrates object transformation and movement.**
- 8. Provides an interactive user experience.**
- 9. Helps students understand graphics programming.**
- 10. Can be extended with advanced features.**

17. LIMITATIONS

- 1. The simulation is primarily a graphical approximation.**
- 2. It may not represent real-world physics with complete accuracy.**
- 3. The project is based on 2D graphics.**
- 4. Advanced 3D rendering is not implemented.**
- 5. Physics calculations are simplified.**
- 6. graphics.h is an older graphics library.**
- 7. Compatibility issues may occur with modern compilers.**
- 8. The application does not provide complete real-time physics simulation.**

18. FUTURE SCOPE

Future improvements may include full 3D simulation, realistic physics calculations, gravity simulation, real-time collision detection, realistic lighting and shadows, metallic reflections, collision sounds, mouse controls, keyboard shortcuts, multiple simulation modes, adjustable ball mass, adjustable gravity, adjustable number of balls, 3D camera rotation, OpenGL-based rendering, performance monitoring, and interactive physics graphs.

The project can also be converted into a modern web-based or 3D application using technologies such as WebGL, Three.js, Unity, or OpenGL.

19. CONCLUSION

The Animated Newton's Cradle project successfully demonstrates the practical application of Computer Graphics concepts through an interactive animation.

The project uses graphical primitives, coordinate systems, transformations, object movement, animation, color, and collision simulation to create a visual representation of Newton's Cradle.

Through this project, we gained practical knowledge about graphics programming and learned how static graphical objects can be converted into interactive animations.

The project also helped us understand the importance of animation in representing real-world concepts visually. The Newton's Cradle simulation provides an effective way to demonstrate the transfer of motion and momentum between objects.

Overall, the project successfully fulfills its objective of creating an educational and interactive Computer Graphics application.

20. REFERENCES

- 1. Computer Graphics concepts and classroom notes.**
- 2. C/C++ Programming reference materials.**
- 3. graphics.h library documentation.**
- 4. Computer Graphics textbooks.**
- 5. Programming tutorials and educational resources.**
- 6. Newton's Cradle physics concepts and collision principles.**